UNIT-3

STRINGS AND LISTS

3.1. STRINGS:

- A string is a sequence of characters.
- In Python, string is a sequence of Unicode character.
- Python strings are "immutable" which means they cannot be changed after they are created.

3.1.1. CREATING STRINGS:

- Creating Strings is easy, and it is done simply by enclosing the characters in single or double quotes.
- The strings in Python consider both single and double quotes as the same.
  Ex:
  String_var = 'Python'
  String_var = "Python"
  String_var = """Python"""

3.1.2. ACCESSING CHARACTERS:

- Python allows to index from the zeroth position in Strings.
- But it also supports negative indexes. Index of '-1' represents the last character of the String. Similarly using '-2' we can access the penultimate element of the string and so on.
  Example:

P	Y	T	H	O	N	–	S	T	R	I	N	G
0	1	2	3	4	5	6	7	8	9	10	11	12
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
sample_str = 'Python String'
print (sample_str[0])        # return 1st character
# output: P
print (sample_str[-1])       # return last character
# output: g
print (sample_str[-2])       # return last second character
# output: n
```

- To retrieve a range of characters in a String we use **'slicing operator'**, the colon **':'**. With the slicing operator, we define the range as **[a:b]**.

```
sample_str = 'Python String'
print (sample_str[3:5])#return a range of character
# ho
print (sample_str[7:])# return all characters from index 7
# String
print (sample_str[:6])# return all characters before index 6
# Python
print (sample_str[7:-4])
# St
```

3.1.3. INVALID USAGE OF STRINGS:

1. If we try to retrieve characters at out of range index then 'IndexError' exception will be raised.

   Ex:
   ```
   sample_str = "Python Supports Machine Learning."
   print (sample_str[1024]) #index must be in range
   # IndexError: string index out of range
   ```

2. String index must be of integer data type. You should not use a float or any other data type for this purpose. Otherwise, the Python subsystem will flag a TypeError exception as it detects a data type violation for the string index.

   Ex:
   ```
   sample_str = "Welcome post"
   print (sample_str[1.25]) #index must be an integer
   # TypeError: string indices must be integers
   ```

3.1.4. STRING OPERATORS

Operator	Operation	Description	Example Code
+	Concatenation	Combining two Strings into one.	var1 = 'Python' var2 = 'String' print (var1+var2) *# PythonString*
*	Repetition	Creates new String by repeating the String given number of times.	var1 = 'Python' print (var1*3) *# PythonPythonPython*
[ ]	Slicing	Prints the character at given index.	var1 = 'Python' print (var1[2]) *# t*
[:]	Range Slicing	Prints the characters present at the given range .	var1 = 'Python' print (var1[2:5]) *# tho*
in	Membership	Returns 'True' value if character is present in the given String.	var1 = 'Python' print ('n' in var1) *# True*
not in	Membership	Returns 'True' value if character is not present in given String.	var1 = 'Python' print ('N' not in var1) *# True*
for	Iterating	Using for we can iterate through all the characters of the String.	var1 = 'Python' for var in var1: print (var) # P # y

Operator	Operation	Description	Example Code
			# t # h # o # n
r/R	Raw String	Used to ignore the actual meaning of Escape characters inside a string. For this we add 'r' or 'R' in front of the String.	print (r'\n') # \n print (R'\n') # \n

3.1.5. STRING FORMATTING OPERATORS:

1. Escape Characters.

- An Escape sequence starts with a backslash (\) which signals the compiler to treat it differently. Python subsystem automatically interprets an escape sequence be it in a single quoted or double quoted Strings.
- Suppose we have a string like –"Python is "widely" used language".
- The double-quote around the word "widely" disguise python that the String ends up there.
- We need a way to tell Python that the double-quotes inside the string are not the string markup quotes. Instead, they are the part of the String and should appear in the output.
- To resolve this issue, we can escape the double-quotes and single-quotes as:

Ex:

```
print ("Python is "widely" used language")
# SyntaxError: invalid syntax
# After escaping with double-quotes
print ("Python is \"widely\" used language")
# Output: Python is "widely" used language
```

Escape Character	Used To Print
\\	Backslash (\)
\"	Double-quote (")
\a	ASCII bell (BEL)
\b	ASCII backspace (BS)
\cx or \Cx	Control-x
\f	ASCII Form feed (FF)
\n	ASCII linefeed (LF)
\N{name}	Character named name in the Unicode database (Unicode only)
\r	Carriage Return (CR)
\t	Horizontal Tab (TAB)
\uxxxx	Character with 16-bit hex value xxxx (Unicode only)
\Uxxxxxxxx	Character with 32-bit hex value xxxxxxxx (Unicode only)
\v	ASCII vertical tab (VT)
\ooo	Character with octal value ooo
\xnn	Character with hex value nn where n can be anything from the range 0-9, a-f or A-F.

2. Python Format Characters '%'

- String '%' operator issued for formatting Strings. This operator is used with print() function.

```
print ("Employee Name: %s,\nEmployee Age:%d" % ('Ashish',25))
# Employee Name: Ashish,
# Employee Age: 25
```

Format Symbol	Conversion
%c	Character
%s	string conversion via str() prior to formatting
%i	signed decimal integer
%d	signed decimal integer
%u	unsigned decimal integer
%o	octal integer
%x	hexadecimal integer (lowercase letters)
%X	hexadecimal integer (UPPER-case letters)
%e	exponential notation (with lowercase 'e')
%E	exponential notation (with UPPER-case 'E')
%f	floating point real number
%g	the shorter of %f and %e
%G	the shorter of %f and %E

3.1.6. STRING METHODS:

Function Name	Description	Example Code
capitalize()	Returns the String with first character capitalized and rest of the characters in lower case.	var = 'PYTHON' print (var.capitalize()) *# Python*
lower()	Converts all the characters of the String to lowercase.	var = 'TechBeamers' print (var.lower()) *# techbeamers*
upper()	Converts all the characters of the String to uppercase.	var = 'TechBeamers' print (var.upper()) *# TECHBEAMERS*
swapcase()	Swaps the case of every character in the String means that lowercase characters are changed to uppercase and vice-versa.	var = 'TechBeamers' print (var.swapcase()) *# tECHbEAMERS*
title()	Returns the 'titlecased' version of String which means that all words start with uppercase and rest of the characters in the words are in lowercase.	var = 'welcome to Python programming' print (var.title()) *# Welcome To Python Programming*
count(str[,beg [,end]])	Returns the number of times substring 'str' occurs in range [beg, end] if beg and end index are given. If it is not given then	var='TechBeamers' str='e' print (var.count(str))

Function Name	Description	Example Code
	substring is searched in whole String. Search is case-sensitive.	*# 3* var1='Eagle Eyes' print (var1.count('e')) *# 2* var2='Eagle Eyes' print (var2.count('E',0,5)) *# 1*
islower()	Returns 'True' if all the characters in the String are in lowercase. If any one character is in uppercase it will return 'False'.	var='Python' print (var.islower()) *# False* var='python' print (var.islower()) *# True*
isupper()	Returns 'True' if all the characters in the String are in uppercase. If any one character is in lowercase it will return 'False'.	var='Python' print (var.isupper()) *# False* var='PYTHON' print (var.isupper()) *# True*
isdecimal()	Returns 'True' if all the characters in String are decimal. If anyone character in the String is of other data-type, it will return 'False'. Decimal characters are those from Unicode category 'Nd'. Complete list of 'Nd' is present at following link: http://www.fileformat.info/info/unicode/category/Nd/list.htm	num=u'2016′ print (num.isdecimal()) *# True*
isdigit()	Returns 'True' for any character for which isdecimal() would return 'True and some characters in 'No' category. If there are any characters other than these, it will return 'False'. Precisely, digits are the characters for which Unicode property includes: Numeric_Type=Digit or Numeric_Type=Decimal. For example, superscripts are digits but fractions not. Complete list of 'No' is present at following link: http://www.fileformat.info/info/unicode/category/No/list.htm	print ('2'.isdigit()) *# True* print ('²'.isdigit()) *# True*

Function Name	Description	Example Code
find(str [,i [,j]])	Searches for 'str' in complete String (if i and j not defined) or in a sub-string of String (if i and j are defined).This function returns the index if 'str' is found else returns '-1'. where, i=search starts from this index j=search ends at this index.	var="Tech Beamers" str="Beam" print (var.find(str)) *# 5* var="Tech Beamers" str="Beam" print (var.find(str,4)) *# 5* var="Tech Beamers" str="Beam" print (var.find(str,7)) *# -1*
replace(old, new[,count])	Replaces all the occurrences of substring 'old' with 'new' in the String. If 'count' is defined then only 'count' number of occurrences of 'old' will be replaced with 'new'. where, old =substring to be replaced new =substring that will replace the old count =number of occurrences of old that will be replaced with new.	var='This is a good example' str='was' print (var.replace('is',str)) *# Thwas was a good exampleprint (var.replace('is',str,1))* *# Thwas is a good example*
split([sep[, maxsplit]])	Returns a list of substring obtained after splitting the String with 'sep' as delimiter. where, sep= delimiter, default is space maxsplit= number of splits to be done	var = "This is a good example" print (var.split()) *# ['This', 'is', 'a', 'good', 'example']print (var.split(' ', 3))* *# ['This', 'is', 'a', 'good example']*
join(seq)	Returns a String obtained after concatenating the sequence 'seq' with a delimiter string. where, seq= sequence of elements to be joined	seq=('ab','bc','cd') str='=' print (str.join(seq)) *# ab=bc=cd*
len(string)	Returns the length of given String	var='This is a good example' print (len(var)) *# 22*

3.1.7. STRING SLICING:

- Given a string s, the syntax for a slice is: s[startIndex : pastIndex]
- The startIndex is the start index of the string. pastIndex is one past the end of the slice.
- **Single Character Slice:** a single character slice S[i] gives i^{th} character of the string.

Ex:

S = 'Hello', S[0] == 'H', S[1] == 'e', S[2] == 'l', S[3] == 'l', S[4] == 'o'

- If you specify a negative index, then it is counted from the end, starting with the number -1. That is, S[-1] == 'o', S[-2] == 'l', S[-3] == 'l', S[-4] == 'e', S[-5] == 'H'.
- **Sub String Slice:** Slice with two parameters S[a:b] returns the substring of length b - a, starting with the character at index a and lasting until the character at index b, not including the last one.
  Ex: S="Hello"
  S[1:4] == 'ell', and you can get the same substring using S[-4:-1]
- Slices with two parameters never cause IndexError
- If you omit the first index, the slice will start from the beginning. If you omit the last index, the slice will go to the end of the string.
  Ex:

```
s = "Hello"
print(s[1:]) # prints "ello"
print(s[:6]) # prints "Hello"
print(s[-4:]) # prints "ello"
```

- Any slice of a string creates a new string and never modifies the original one.
- **Subsequence Slice:** If you specify a slice with three parameters S[a:b:d], the third parameter specifies the step, same as for function range(). In this case only the characters with the following index are taken: a a + d, a + 2 * d and so on, until and not including the character with index b. If the third parameter equals to 2, the slice takes every second character, and if the step of the slice equals to -1, the characters go in reverse order. For example, you can reverse a string like this: S[::-1].
  Ex:

```
s = "Hello"
print(s[0:5:2]) # prints "Hlo"
```

3.2. LISTS:

- Python lists are ordered sequences of items. For instance, a sequence of *n* numbers might be called *S*:
  $S = s_0, s_1, s_2, s_3, \ldots, s_{n-1}$
- A list or array is a sequence of items where the entire sequence is referred to by a single name (i.e. s) and individual items can be selected by indexing (i.e. s[i]).
- Python lists are dynamic. They can grow and shrink on demand.
- Python lists are also heterogeneous, a single list can hold arbitrary data types.
- Python lists are mutable sequences of arbitrary objects.

3.2.1. CREATING A LIST:

- In Python programming, a list is created by placing all the items (elements) inside a square bracket [ ], separated by commas.
- It can have any number of items and they may be of different types (integer, float, string etc.).

```
# empty list
```

```
my_list = []

# list of integers
my_list = [1, 2, 3]

# list with mixed datatypes
my_list = [1, "Hello", 3.4]
```

- Also, a list can even have another list as an item. This is called nested list.

```
# nested list
my_list = ["mouse", [8, 4, 6], ['a']]
```

- **List() Method To Create A List**
  - Python includes a built-in list() method.
  - It accepts either a sequence or tuple as the argument and converts into a Python list.

    Ex:

```
theList = list() #empty list
len(theList)
0
```

3.2.2. ACCESSING ELEMENTS OF A LIST:

There are various ways in which we can access the elements of a list.

1. List Index

- We can use the index operator [] to access an item in a list. Index starts from 0. So, a list having 5 elements will have index from 0 to 4.

  Ex:

```
my_list = ['p','r','o','b','e']
print(my_list[0])# Output: p
print(my_list[2])# Output: o
print(my_list[4])# Output: e
```

- Trying to access an element other that this will raise an IndexError. The index must be an integer. We can't use float or other types, this will result into TypeError.

  Ex:

```
# my_list[4.0]# Error! Only integer can be used for indexing
```

- Nested list are accessed using nested indexing.

  Ex:

```
n_list = ["Happy", [2,0,1,5]]
print(n_list[0][1])# Output: a
print(n_list[1][3])# Output: 5
```

- Python allows negative indexing (Reverse Indexing) for its sequences. The index of -1 refers to the last item, -2 to the second last item and so on.

  Ex:

```
my_list = ['p','r','o','b','e']
# Output: e
print(my_list[-1])
# Output: p
print(my_list[-5])
```

3.2.3. OPERATIONS ON LISTS:

- **Concatenation:**
  We can concatenate lists by using + or extend method.
  Example:
  ```
  list_a = [1, 2, 3, 4]
  list_b = [5, 6, 7, 8]
  list_c = list_a + lis_b
  print list_c # Output: [1, 2, 3, 4, 5, 6, 7, 8]
  ```

  Extend operator can be used to concatenate one list into another. It is not possible to store the value into the third list. One of the existing list has to store the concatenated result.

  Example:
  ```
  list_c = list_a.extend(list_b)
  print list_c # Output: NoneType
  print list_a # Output: [1, 2, 3, 4, 5, 6, 7, 8]
  print list_b # Output: [5, 6, 7, 8]
  ```

- **Slicing:**
  - Given a string s, the syntax for a slice is:
    s[startIndex : pastIndex]
  - The startIndex is the start index of the string. pastIndex is one past the end of the slice.
    Ex:
    theList = [1, 2, 3, 4, 5, 6, 7, 8]
    theList[2:5] #[3,4,5]
    Since Python list follows the zero-based index rule, so the first index starts at 0.
  - In slicing, if you leave the start, then it means to begin slicing from the 0th index.
    Ex:
    theList[:2]
    [1, 2]
  - While slicing a list, if the stop value is missing, then it indicates to perform slicing to the end of the list. It saves us from passing the length of the list as the ending index.
    Ex:
    theList[2:]
    [3, 4, 5, 6, 7, 8]
  - **Reverse A Python List Using The Slice Operator**
    - It is effortless to achieve this by using a special slice syntax (::-1). But please remember that reversing a list this way consumes more memory than an in-place reversal.
    - Here, it creates a shallow copy of the Python list which requires long enough space for holding the whole list.
      Ex:
      theList[::-1]
      [8, 7, 6, 5, 4, 3, 2, 1]

- **Iteration:**
  - Python provides a traditional for-in loop for iterating the list. The for statement makes it super easy to process the elements of a list one by one.

    for element in theList:
    print(element)
  - If you wish to use both the index and the element, then call the enumerate() function.

    for index, element in enumerate(theList):
    print(index, element)
  - If you only want the index, then call the range() and len() methods.

    for index in range(len(theList)):
    print(index)

  Ex:

```
theList = ['Python', 'C', 'C++', 'Java', 'CSharp']

for language in theList:
 print("I like", language)
```

Output –

I like Python
I like C
I like C++
I like Java
I like CSharp

List methods:

- **list.append(elem)** -- adds a single element to the end of the list. Common error: does not return the new list, just modifies the original.
- **list.insert(index, elem)** -- inserts the element at the given index, shifting elements to the right.
- **list.extend(list2)** -- adds the elements in list2 to the end of the list. Using + or += on a list is similar to using extend().
- **list.index(elem)** -- searches for the given element from the start of the list and returns its index. Throws a ValueError if the element does not appear (use "in" to check without a ValueError).
- **list.remove(elem)** -- searches for the first instance of the given element and removes it (throws ValueError if not present)
- **list.sort()** -- sorts the list in place (does not return it). (The sorted() function shown later is preferred.)
- **list.reverse()** -- reverses the list in place (does not return it)
- **list.pop(index)** -- removes and returns the element at the given index. Returns the rightmost element if index is omitted (roughly the opposite of append()).
- **Example**:

```
list = ['larry', 'curly', 'moe']
list.append('shemp')          ## append elem at end
list.insert(0, 'xx')         ## insert elem at index 0
list.extend(['yy', 'zz'])  ## add list of elems at end
print(list) #['xx','larry','curly','moe','shemp','yy','zz']
print(list.index('curly'))     ## 2
```

functions of List

Function	Description
all()	Return True if all elements of the list are true (or if the list is empty). lst1=[1,2,3,4] lst2=[10,11,0,14] print(all(lst1)) # True print(all(lst2)) # False
any()	Return True if any element of the list is true. If the list is empty, return False. lst=[1,2,0,4] print(any(lst)) # True
len()	Return the length (the number of items) in the list. Print(len(lst)) # 4
list()	Convert an iterable (tuple, string, set, dictionary) to a list. t=(1,2,3,4) print(list(t)) # [1,2,3,4]
max()	Return the largest item in the list. print(max(lst)) # 4
min()	Return the smallest item in the list print(min(lst)) # 0
sorted()	Return a new sorted list (does not sort the list itself). my_list=[6,3,8,4,1] print(sorted(my_list)) # [1,3,4,6,8]
sum()	Return the sum of all elements in the list. print(sum(my_list)) # 22

Deletion of List Elements

To Delete one or more elements, i.e. remove an element, many built in functions can be used, such as pop() & remove() and keywords such as del.

- **pop():** Index is not a necessary parameter, if not mentioned takes the last index.
  Syntax:

```
list.pop([index])
```

Note: Index must be in range of the List, elsewise IndexErrors occurs.

```
List = [2.3, 4.445, 3, 5.33, 1.054, 2.5]
print(List.pop())
```

Output:

```
2.5
```

```
List = [2.3, 4.445, 3, 5.33, 1.054, 2.5]
print(List.pop(0))
```

Output:

```
2.3
```

- **del() :** Element to be deleted is mentioned using list name and index.
  Syntax:

```
del list.[index]
```

```
List = [2.3, 4.445, 3, 5.33, 1.054, 2.5]
del List[0]
print(List)
```

Output:

```
[4.445, 3, 5.33, 1.054, 2.5]
```

- **remove():** Element to be deleted is mentioned using list name and element.
  Syntax:

```
list.remove(element)
```

```
List = [2.3, 4.445, 3, 5.33, 1.054, 2.5]
List.remove(3)
print(List)
```

Output:

```
[4.445, 5.33, 1.054, 2.5]
```